表 7.2: PID 分类与各传输类型下的作用

类别	PID	含义与在本项目中的位置
Token	`OUT`	主机 → 设备：后面跟一个主机写往该端点的 Data 包（EP0 的 control OUT、ch4 之外的下发命令）
Token	`IN`	主机 → 设备：请求该端点回一个 Data 包。ch6 键盘每隔 `bInterval` 发一次 IN，ch4 MT7601U 的 bulk-IN 也是 IN
Token	`SETUP`	只在端点 0，启动一次 control 传输的 SETUP 阶段（枚举、`SET_CONFIGURATION` 等）
Token	`PING`	仅高速 bulk OUT：先探测端点是否还有缓冲，再决定发不发 Data，避免把数据丢进满的端点
Data	`DATA0/DATA1`	数据包，二者交替出现做 PID 序列翻转（toggle），用来发现丢包或重复包；EP0 的 SETUP 恒用 DATA0
Handshake	`ACK`	接收方 → 发送方：已正确收到
Handshake	`NAK`	设备 → 主机：“收到了但暂时不能传”——IN 端点没数据、或 OUT 端点缓冲满。是流控不是错误，主机下一帧重试
Handshake	`NYET`	高速设备 → 主机：“收到了但还没准备好”，与 NAK 区分；split 事务用它携带本帧是否完成的信息
Handshake	`STALL`	设备 → 主机：端点被软件停止（halt），不是重试能解决的，要驱动显式清除

相包在高速 Hub 的事务转换器（TT）处拆成两拍：主机先发一个 SSPLIT 触发 TT 去总线上跑完整事务，隔 1 ~ 2 个微帧再发 CSPLIT 把结果（DATA、ACK、NAK 或 NYET）取回来。

规范按“可靠性、时限、方向”把这同一套三相事务组合成**四种传输类型**（表 7.3）。前三者都靠 Handshake 与 DATA0/DATA1 翻转保证不丢不错，只有 isochronous 干脆去掉握手、发出去就不管，用可靠性换实时性；bulk 与 interrupt 同样可靠，区别只落在“有没有周期承诺”这一条上。本项目的落点很清楚：枚举走 control（ch0），键盘和 Hub 状态走 interrupt（ch6、ch7），MT7601U 网卡的报文走 bulk（ch4），isochronous 未使用。

表 7.3: USB 四种传输类型，以及它们在本项目里落在哪个 channel

类型	可靠性与时限	典型数据	本项目
Control	保证送达；不承诺延迟但优先调度，主机发起、双向	枚举、发命令、取描述符	ch0（EP0）
Interrupt	保证送达；有周期上限（`bInterval` 轮询），每包很小	键盘按键、鼠标、Hub 状态变化	ch6（键盘）、ch7（Hub）
Bulk	保证送达；无时限，只在剩余带宽里搬运，量大	U 盘、打印、网络报文	ch4（MT7601U bulk-IN/OUT）
Isochronous	不重传、无握手；预留带宽、严格实时	音频、视频流	（本项目未用）

7.4.3 描述符：设备、配置、接口与端点

USB 设备用一组描述符介绍自己（图 7.6）。每个描述符的第 0 字节是长度 `bLength`，第 1 字节是类型。层次是：一个设备有若干配置（几乎总是 1 个）；一个配置包含若干接口，每个接口是一项独立的功能，有自己的 class/subclass/protocol；一个接口有若干端点，端点是真正收发数据的地方，有编号、方向、传输类型、最大包长和轮询间隔。端点 0 是所有设备都有的默认控制端点，枚举全靠它。

很多键盘接收器是复合设备：接口 0 是键盘，接口 1 是鼠标或多媒体键。所以驱动不能看设备描述符的 class（它通常是 0，意思是“看各个接口”），也不能凭 VID/PID 白名单，而要遍历配置描述符：

```
// GET_DESCRIPTOR(Configuration) 先取 9 字节拿到 wTotalLength，再一次取全部
for(int off = 0; off + 9 <= total && ctrl_buf[off] >= 2; off += ctrl_buf[off]){
  if(ctrl_buf[off + 1] == 4 &&                    // 接口描述符
     ctrl_buf[off + 5] == 3 &&                    // class: HID
     ctrl_buf[off + 6] == 1 &&                    // subclass: Boot Interface
     ctrl_buf[off + 7] == 1){                     // protocol: Keyboard
    sel->interface_number = ctrl_buf[off + 2];
    return 1;
```